

SAHYADRI SAMRAKSHANE (FOREST SENTINEL)

Complete AI Development Package

Screen Prompts + Navigation Map + Complete SOP for AI
Code Generator



PART 1: INDIVIDUAL

SCREEN PROMPTS

SCREEN 1: Splash & Onboarding

Screen ID: SCR-001-SPLASH

AI Prompt:

text

Create a Jetpack Compose splash screen for "Sahyadri Samrakshane" (Forest Sentinel) environmental app with:

VISUAL DESIGN:

- Background: Gradient from deep forest green (#1B5E20) to lighter green (#4CAF50)
- Center: Animated app logo (tree icon with shield overlay)
- App name: "Sahyadri Samrakshane" in bold, nature-inspired font
- Tagline: "Protect the Forest. Protect the Future." below logo
- Animation: Logo scales in with fade (300ms duration)

FUNCTIONALITY:

- Auto-navigate to onboarding (first launch) or home (returning user) after 2 seconds
- Check SharedPreferences key: "is_first_launch"
- Display app version number at bottom (small text)

TECHNICAL REQUIREMENTS:

- Use Material 3 theming with custom green color scheme
- Implement with LaunchedEffect for navigation delay
- ViewModel to check first launch status
- Navigation to either OnboardingScreen or HomeScreen

COLOR PALETTE:

- Primary: #2E7D32 (Forest Green)
- Secondary: #66BB6A (Light Green)
- Background: Gradient #1B5E20 → #4CAF50
- Text: #FFFFFF (White)

CODE STRUCTURE:

- SplashScreen composable
- SplashViewModel with checkFirstLaunch() method

- Navigation controller integration

SCREEN 2: Onboarding Flow (3 Screens)

Screen ID: SCR-002-ONBOARDING

AI Prompt:

text

Create a 3-screen onboarding flow using Jetpack Compose HorizontalPager with:

SCREEN 1 - WELCOME:

- Large illustration: Forest with mountains (use vector drawable)
- Title: "Protect the Western Ghats"
- Description: "Report forest fires, illegal logging, landslides, and wildlife threats instantly to forest authorities"
- Skip button (top-right)
- Next button (bottom)

SCREEN 2 - CAMERA PERMISSION:

- Illustration: Camera icon with forest background
- Title: "Capture Evidence"
- Description: "Take high-resolution photos of environmental threats to help authorities respond quickly"
- Rationale: "We need camera access to document alerts"
- Grant Permission button
- Skip button

SCREEN 3 - LOCATION PERMISSION:

- Illustration: GPS pin on map
- Title: "Precise Location Matters"
- Description: "GPS coordinates help forest teams reach the exact spot within minutes"
- Rationale: "We need location access to tag alerts accurately"
- Grant Permission button
- Get Started button (navigates to Home)

FUNCTIONALITY:

- HorizontalPager with page indicators (dots)
- Permission requests using `accompanist-permissions` library
- Save onboarding completion to `SharedPreferences`
- Smooth page transitions

TECHNICAL REQUIREMENTS:

- Use `rememberPermissionState` for runtime permissions
- `OnboardingViewModel` to handle permission results
- `NavigationController` to move to `HomeScreen`
- Material 3 buttons with custom green theme

COLOR SCHEME:

- Background: White with green accents
 - Buttons: Solid #2E7D32 (primary green)
 - Text: Dark gray #212121 for readability
-
- Illustrations: Earth-tone palette

SCREEN 3: Home Dashboard

Screen ID: SCR-003-HOME

AI Prompt:

text

Create a Jetpack Compose Home Dashboard for ecological alert app with:

TOP SECTION:

- Custom TopAppBar with gradient background (#2E7D32 to #388E3C)
- App logo (small) + "Sahyadri Samrakshane" title
- Connectivity status indicator (cloud icon: green=online, red=offline)
- Profile/Settings icon (top-right)

ALERT STATS CARDS (Grid 2x2):

Card 1: Total Alerts Submitted

- Icon: 📊 (chart)
- Number: Large display (e.g., "12")
- Label: "My Reports"
- Background: Light green (#E8F5E9)

Card 2: Verified Alerts

- Icon: ✅ (checkmark)
- Number: e.g., "8"
- Label: "Verified"
- Background: Light blue (#E3F2FD)

Card 3: Pending Alerts

- Icon: ⌚ (hourglass)
- Number: e.g., "4"
- Label: "Pending Review"
- Background: Light orange (#FFF3E0)

Card 4: Community Impact

- Icon: 🌳 (tree)
- Number: e.g., "247" (district-wide)
- Label: "Total Community Alerts"
- Background: Light teal (#E0F2F1)

QUICK REPORT SECTION:

- Large FAB (Floating Action Button) at bottom-center
- Icon: Camera + Location pin combined
- Text: "Report Alert"

- Color: Vibrant green (#43A047)
- Ripple effect on press
- Navigates to Alert Type Selection screen

RECENT ALERTS LIST:

- Scrollable LazyColumn showing last 5 alerts
- Each item shows:
 - Alert type icon (🔥 fire, 🪵 logging, 🏔️ landslide, 🦌 wildlife)
 - Alert title + location (truncated)
 - Timestamp (relative: "2 hours ago")
 - Status badge (color-coded)
- Tap to view full alert details

EDUCATION MODULE ENTRY:

- Small card at bottom: "Learn Eco-Safe Practices"
- Icon: 📖
- Tap navigates to Education Screen

FUNCTIONALITY:

- Pull-to-refresh to sync with Firebase
- Real-time status updates via Firestore listeners
- WorkManager background sync indicator
- Empty state: "No alerts yet. Be the first to protect the forest!"

TECHNICAL REQUIREMENTS:

- HomeViewModel with LiveData/StateFlow
- Repository pattern fetching from Room + Firestore
- Offline-first: display Room data, sync with Firebase in background
- Navigation to: ReportAlertScreen, AlertDetailsScreen, EducationScreen, SettingsScreen

DATA FLOW:

- On launch: Load cached alerts from Room DB
- Background: Sync with Firestore, update Room
- Display: Observe Room DB LiveData for reactive UI

SCREEN 4: Alert Type Selection

Screen ID: SCR-004-ALERT-TYPE

AI Prompt:

text

Create a Jetpack Compose Alert Type Selection screen with:

HEADER:

- Title: "What did you witness?"
- Subtitle: "Select the type of ecological threat"
- Back button (top-left)

ALERT TYPE CARDS (4 options in 2x2 grid):

1. FOREST FIRE

- Large icon: 🔥 (fire emoji or custom vector)
- Title: "Forest Fire"
- Description: "Active flames or smoke"
- Background gradient: Red to orange (#D32F2F → #FF6F00)
- Border: 2dp solid when selected

2. LANDSLIDE

- Icon: 🗑️ (mountain with falling rocks)
- Title: "Landslide"
- Description: "Ground collapse or erosion"
- Background: Brown to gray (#5D4037 → #757575)

3. ILLEGAL TREE CUTTING

- Icon: 🪓 (axe + tree stump)
- Title: "Illegal Logging"
- Description: "Unauthorized tree felling"
- Background: Dark brown (#3E2723 → #6D4C41)

4. WILDLIFE SIGHTING

- Icon: 🦌 (deer or generic wildlife)
- Title: "Wildlife Threat"
- Description: "Poaching or unusual activity"
- Background: Green to teal (#388E3C → #00897B)

INTERACTION:

- Single selection (radio button behavior)
- Card elevation increases on selection
- Selected card: highlighted border + scale animation
- Unselected: 50% opacity

BOTTOM BUTTON:

- "Next: Capture Photo" button (full-width)
- Enabled only when a type is selected
- Navigates to Camera Capture screen with selected type as argument

TECHNICAL REQUIREMENTS:

- State management: `remember { mutableStateOf<AlertType?>(null) }`
- AlertType enum: `FOREST_FIRE, LANDSLIDE, ILLEGAL_LOGGING, WILDLIFE_SIGHTING`
- Navigation with type argument: `navController.navigate("camera/${selectedType}")`
- Material 3 Card with `clickable` modifier
- Accessibility: Content descriptions for TalkBack

UI ENHANCEMENTS:

- Subtle parallax effect on card hover (if supported)
- Haptic feedback on selection
- Smooth enter animation (cards fade + slide in sequentially)

SCREEN 5: Camera Capture with GPS

Screen ID: SCR-005-CAMERA

AI Prompt:

text

Create a Jetpack Compose CameraX integration screen with real-time GPS display:

CAMERA VIEWFINDER:

- Full-screen CameraX Preview
- Use ProcessCameraProvider with front/back camera toggle
- Image capture with high-resolution settings
- Flash toggle button (top-left)
- Camera flip button (top-right)

GPS OVERLAY (Bottom overlay card):

- Semi-transparent dark background (#000000 80% opacity)
- Icon: 📍 (location pin)
- Text: "📍 Lat: 15.4909, Lon: 73.8278" (live updating)
- Accuracy indicator: Green circle if <10m, yellow if 10-50m, red if >50m
- Status text: "GPS Accuracy: 8 meters"

CONTROLS:

- Large circular capture button (bottom-center)
- Icon: Camera shutter
- Ripple animation on press
- Disabled if GPS accuracy >100m (show warning toast)

POST-CAPTURE PREVIEW:

- Show captured image in full screen
- GPS coordinates frozen at capture moment
- "Retake" button (left)
- "Use This Photo" button (right)
- Confirm navigates to Add Details screen

GPS LOCATION LOGIC:

- Use FusedLocationProviderClient for location
- Request high-accuracy location updates
- Update UI every 2 seconds while on screen
- Capture location at exact moment of photo capture
- Store lat/lon in photo metadata (EXIF) if possible

PERMISSIONS HANDLING:

- Check camera permission before showing viewfinder
- Check location permission before GPS display
- Show rationale dialog if permission denied
- Fallback: Allow photo without GPS (show warning)

TECHNICAL REQUIREMENTS:

- CameraViewModel with:
 - capturePhoto(onSuccess: (File, LatLng) -> Unit)
 - startGPSUpdates()
 - stopGPSUpdates()
- Use coroutines for async camera/GPS operations
- Save photo to app-specific external storage (getExternalFilesDir)
- Compress image to <2MB before saving

ERROR STATES:

- No GPS signal: "Waiting for GPS lock..." (yellow banner)
- Camera error: "Camera unavailable" (red banner)
- Low accuracy: "Move to open area for better GPS" (toast)

UI/UX DETAILS:

- GPS overlay slides up from bottom with animation
 - Accuracy circle pulses when acquiring signal
 - Haptic feedback on successful capture
-
- Loading indicator while processing image

SCREEN 6: Add Alert Details

Screen ID: SCR-006-ADD-DETAILS

AI Prompt:

text

Create a Jetpack Compose form screen for adding alert details with AI assistance:

HEADER:

- Alert type badge (color-coded based on selection)
- Title: "Add Alert Details"
- Captured photo thumbnail (top, clickable to view full)
- GPS coordinates display (small text below photo)

AI CLASSIFICATION SECTION:

- Loading indicator: "📷 AI is analyzing your photo..."
- Result card (after analysis):
 - "AI detected: Forest Fire with 87% confidence"
 - Scene description: "Smoke visible rising from dense forest area. High severity."
 - "Is this correct?" with Yes/No buttons
 - If No → Manual alert type selection dropdown

FORM FIELDS:

1. Alert Title (Optional):

- TextInputLayout with hint: "Brief description (optional)"
- Max 50 characters
- Placeholder: "e.g., Large fire near hilltop"

2. Detailed Description (Optional):

- Multi-line TextInputLayout
- Hint: "What exactly did you see? (optional)"
- Max 200 characters
- Placeholder: "Add any additional details..."

3. Is it Active/Ongoing?

- Toggle switch: "Threat is currently active"
- Default: ON

4. Location Confirmation:

- Map preview showing GPS pin (Google Maps or OSM)
- "Location accurate?" with Yes/No
- If No → Manual map picker

5. Photo Preview:

- Thumbnail with "Change Photo" button
- Compression status: "Photo size: 1.2 MB ✓"

GEMINI AI INTEGRATION:

- Call Gemini Vision API on screen load
- Send captured photo (base64 or URI)
- Prompt: "Analyze this image for environmental threats. Classify as: Forest Fire, Landslide, Illegal Logging, or Wildlife. Provide confidence score and brief scene description."
- Parse JSON response and display
- Show "AI unavailable" if offline (graceful fallback)

BOTTOM ACTIONS:

- "Submit Alert" button (primary, full-width)
 - Disabled until AI analysis completes OR user manually selects type
 - Shows loading spinner during Firebase upload
- "Cancel" button (text button)

VALIDATION:

- At least one of: AI classification OR manual type selection
- GPS coordinates must be present
- Photo must be attached
- If offline: Show "Will sync when online" badge

TECHNICAL REQUIREMENTS:

- AddDetailsViewModel with:
 - analyzePhoto(imageUri): Flow<AIResult>
 - submitAlert(alert: Alert): Result<AlertId>
 - uploadPhotoToStorage(imageFile): String (URL)
- Gemini API call with try-catch error handling
- Firebase Storage upload with progress listener
- Firestore document creation with batch write

DATA MODEL:

```
Alert(  
    id: String,  
    type: AlertType,  
    photoUrl: String,  
    latitude: Double,  
    longitude: Double,  
    title: String?,  
    description: String?,  
    isActive: Boolean,  
    aiConfidence: Float?,  
    timestamp: Long,  
    status: AlertStatus.REPORTED,  
    userId: String?  
)
```

UI ENHANCEMENTS:

- AI analysis card animates in with fade + slide
- Success animation on submit (green checkmark)
- Error Snackbar if upload fails
- Offline mode indicator (yellow banner at top)

SCREEN 7: Submission Confirmation

Screen ID: SCR-007-CONFIRMATION

AI Prompt:

text

Create a Jetpack Compose success confirmation screen with:

VISUAL DESIGN:

- Animated checkmark (green circle with checkmark scaling in)
- Lottie animation (optional): Forest with protective shield
- Background: Gradient light green (#E8F5E9 → #C8E6C9)

SUCCESS MESSAGE:

- Large title: "Alert Submitted Successfully!"
- Subtitle: "Thank you for protecting the Western Ghats"

ALERT DETAILS CARD:

- Report ID: "FS-2024-1847" (generated unique ID)
- Alert Type: Icon + label (e.g., 🔥 Forest Fire)
- Location: Truncated lat/lon with "View on Map" link
- Timestamp: "Submitted on Jan 15, 2024 at 3:42 PM"
- Status: "Reported" badge (orange)

RESPONSE TIMELINE (Estimated):

- Card with timeline graphic
- Step 1: "Reported" ✅ (Done)
- Step 2: "Verification" ⌚ (In progress - 10-30 min)
- Step 3: "Team Dispatched" 🚒 (Pending)
- Estimated response: "Forest team alerted. Expected arrival: 45-60 min"

ACTIONS:

- "View My Alerts" button (primary)
 - Navigates to Alert History screen
- "Report Another Alert" button (secondary)
 - Navigates back to Alert Type Selection
- "Share Alert" button (icon button)
 - Share report ID + location via system share sheet
 - Text: "I reported a [type] alert via Sahyadri Samrakshane app. Report ID: FS-2024-1847"

NOTIFICATION SETUP:

- "Get Updates" switch
- Text: "Notify me when status changes"
- If enabled: Subscribe to FCM topic for this alert ID

EDUCATIONAL TIP (Bottom card):

- Random eco-tip from knowledge base
- Icon: 💡
- Example: "Tip: Never attempt to fight forest fires yourself. Always maintain safe distance."

TECHNICAL REQUIREMENTS:

- ConfirmationViewModel with:
 - getAlertById(id): Flow<Alert>
 - subscribeToNotifications(alertId)
- Navigation: Pass alert ID as argument
- FCM topic subscription for real-time status updates
- Share intent for report sharing

ANIMATIONS:

- Entry: Checkmark scales in + bounces
- Alert card: Slides up from bottom
- Timeline: Sequential reveal (stagger 200ms each)

AUTO-NAVIGATION:

- After 10 seconds of inactivity: Show "Returning to home..." toast
- Auto-navigate to Home screen after 12 seconds
- User action cancels auto-navigation

SCREEN 8: Alert History / My Reports

Screen ID: SCR-008-ALERT-HISTORY

AI Prompt:

text

Create a Jetpack Compose Alert History screen with filtering and search:

HEADER:

- Title: "My Alerts"
- Total count badge: "12 reports"
- Filter icon (top-right) → Opens filter bottom sheet
- Search icon → Expands search bar

FILTER CHIP ROW:

- Horizontal scrollable chips:
 - "All" (default selected)
 - "Reported" (orange)
 - "Verified" (blue)
 - "Team Dispatched" (green)
 - "Resolved" (gray)
- Selection updates list filter

ALERT LIST (LazyColumn):

Each item card shows:

- Alert type icon (large, left side)
- Alert title (or auto-generated: "Forest Fire Alert")
- Location snippet: "Near Amboli, Sindhudurg"
- Timestamp: "2 days ago"
- Status badge (color-coded, right side)
- Photo thumbnail (small, bottom-left)
- Tap → Navigate to Alert Detail screen

GROUPING:

- Section headers: "This Week", "Last Month", "Older"
- Sticky headers while scrolling

EMPTY STATES:

- No alerts: Illustration + "No alerts yet. Start protecting the forest!"

- No results for filter: "No [status] alerts found"

SWIPE ACTIONS:

- Swipe right: "View on Map" (blue)
- Swipe left: "Share" (green)

PULL-TO-REFRESH:

- Pull down to sync with Firebase
- Show loading indicator at top
- Update status badges if changed

SEARCH FUNCTIONALITY:

- Search by alert type, location keywords, or date
- Debounced search (300ms delay)
- Highlight matching text in results

BOTTOM SHEET FILTER:

Advanced filters:

- Date range picker
- Alert type multi-select
- Status multi-select
- Location radius slider (km from current location)
- "Apply Filters" button

TECHNICAL REQUIREMENTS:

- AlertHistoryViewModel with:
 - getAllAlerts(): Flow<List<Alert>>
 - filterAlerts(status: AlertStatus?): Flow<List<Alert>>
 - searchAlerts(query: String): Flow<List<Alert>>
- Combine Room DB + Firestore data
- Paging 3 library for large lists (if >100 alerts)
- Firestore real-time listener for status updates

DATA SYNC:

- Display Room DB data immediately (offline support)
- Background sync with Firestore
- Show "Syncing..." indicator when online
- Conflict resolution: Firestore is source of truth

UI DETAILS:

- Status badge colors:
 - Reported: #FF9800 (Orange)
 - Verified: #2196F3 (Blue)
 - Team Dispatched: #4CAF50 (Green)
 - Resolved: #9E9E9E (Gray)
- Card elevation: 2dp
- Item animator for smooth insertions/deletions

SCREEN 9: Alert Detail View

Screen ID: SCR-009-ALERT-DETAIL

AI Prompt:

text

Create a Jetpack Compose full alert detail screen with map and status tracking:

HEADER IMAGE:

- Full-width alert photo (aspect ratio 16:9)
- Pinch-to-zoom enabled
- Status badge overlay (top-right of photo)
- Back button (top-left)

ALERT INFO SECTION:

1. Type & Title:

- Large alert type icon + name
- User-provided title (if any)
- Report ID (small, copyable)

2. Status Timeline:

- Horizontal stepper showing:
 - Reported → Verified → Team Dispatched → Resolved
- Each step with timestamp
- Current step highlighted
- Future steps grayed out

3. Location Details:

- Embedded map (Google Maps or OSM)
- Pin at exact GPS coordinates
- Lat/Lon displayed below map
- "Get Directions" button → Opens Google Maps
- Address reverse geocoding: "Amboli Forest, Sindhudurg District"

4. AI Analysis (if available):

- Card titled "AI Insights"
- Scene description from Gemini
- Confidence score: "87% Forest Fire"
- Severity level: High/Medium/Low

5. User Description:

- Text field showing user's notes
- "Active threat" indicator if flagged

6. Metadata:

- Submitted by: Anonymous (or user name if auth)
- Submission time: Full timestamp
- Last updated: Relative time
- View count (if tracking enabled)

FOREST DEPARTMENT RESPONSE (if verified):

- Card titled "Official Response"
- Verification notes from forest guard
- Team dispatch details:
 - Assigned team ID
 - Estimated arrival time
 - Contact number (if critical)

ACTIONS SECTION:

- "Share Alert" button → System share sheet
- "Report Issue" button (if alert is incorrect)
- "Mark as Resolved" (if user is reporter + status allows)

SIMILAR ALERTS NEARBY:

- "Nearby Alerts" section (if within 5km)

- Horizontal scrollable cards
- Tap to view other alert details

TECHNICAL REQUIREMENTS:

- AlertDetailViewModel with:
 - getAlertById(id): Flow<Alert>
 - updateAlertStatus(id, newStatus)
 - getNearbyAlerts(lat, lon, radiusKm): List<Alert>
- Firestore real-time listener for this alert
- Map integration: Google Maps Compose or OSM
- Reverse geocoding API for address

REAL-TIME UPDATES:

- Listen to Firestore document changes
- Show toast when status updates: "Alert verified by forest officials"
- Animate status timeline progression

UI ENHANCEMENTS:

- Parallax effect on header image scroll
- Skeleton loading for slow network
- Error state if alert not found

- Offline banner if no internet

SCREEN 10: Education Module

Screen ID: SCR-010-EDUCATION

AI Prompt:

text

Create a Jetpack Compose educational content screen with:

HEADER:

- Title: "Eco-Safe Practices"
- Subtitle: "Protect yourself and the forest"
- Illustration: Forest with hikers (vector drawable)

CONTENT CATEGORIES (Tabs or sections):

1. FOREST FIRE SAFETY:

- Icon: 🔥
- Tips list:
 - ✓ Never light fires in eco-sensitive zones
 - ✓ Extinguish cigarettes completely
 - ✓ Report smoke immediately
 - ✓ Maintain 100m distance from active fires
 - ✓ Move perpendicular to wind direction if escaping
- Infographic: Wind direction + escape route diagram

2. LANDSLIDE AWARENESS:

- Icon: 
- Tips:
 - ✓ Avoid trekking during heavy monsoon
 - ✓ Watch for ground cracks and tilted trees
 - ✓ Stay away from steep slopes after rain
 - ✓ Alert authorities if you see new cracks
- Video embed (optional): "Recognizing landslide warning signs"

3. ILLEGAL LOGGING REPORTING:

- Icon: 
- Tips:
 - ✓ Note the location and time
 - ✓ Take photos from a safe distance
 - ✓ Do NOT confront loggers
 - ✓ Report to forest department immediately
 - ✓ Provide vehicle details if visible

4. WILDLIFE SAFETY:

- Icon: 
- Tips:
 - ✓ Never approach wild animals
 - ✓ No flash photography
 - ✓ Report poaching evidence immediately
 - ✓ Keep noise levels low in wildlife zones
 - ✓ Maintain 30m distance from elephants
- Emergency contacts card

5. GENERAL ECO-ETHICS:

- Icon: 
- Do's and Don'ts:
 - ✓ Carry back all waste
 - ✓ Use designated trails only
 - ✓ No plastic in forest areas
 - ✓ Respect tribal communities
 - ✗ Don't pick flowers or break branches
 - ✗ Don't play loud music

INTERACTIVE QUIZ (Bottom section):

- "Test Your Knowledge" button
- 5-question quiz on eco-practices
- Gamification: Earn "Forest Guardian" badge

DOWNLOADABLE RESOURCES:

- "Download Offline Guide" button (PDF)
- Saves to device for offline reading
- 2MB compressed PDF with all tips

TECHNICAL REQUIREMENTS:

- EducationViewModel with:
 - getTipsByCategory(category): List<EcoTip>
 - markTipAsRead(tipId)
- Local storage: Room DB for offline access
- PDF download: DownloadManager API
- Quiz state management with scoring

UI DESIGN:

- Expandable/collapsible sections (accordion)
- Icons for each tip (vector drawables)
- Green checkmarks for completed tips
- Progress bar: "You've read 7/15 tips"

ACCESSIBILITY:

- Text-to-speech support for tips

- High contrast mode for readability
- Large touch targets (48dp minimum)

SCREEN 11: Settings & Profile

Screen ID: SCR-011-SETTINGS

AI Prompt:

text

Create a Jetpack Compose settings screen with:

PROFILE SECTION (if auth enabled):

- Profile icon placeholder
- "Anonymous User" (default)
- "Login to save progress" button
- Stats: "12 alerts submitted, 8 verified"

GENERAL SETTINGS:

1. Notifications:

- "Alert Status Updates" toggle (default: ON)
- "Community Milestones" toggle
- "Educational Tips" toggle

2. Location Preferences:

- "High Accuracy GPS" toggle (default: ON)
- "Save Battery Mode" (lower accuracy)
- GPS accuracy info text

3. Photo Quality:

- Dropdown: High (default) / Medium / Low
- Storage impact info

4. Theme:

- Radio buttons: Light / Dark / System Default
- Preview swatch

5. Language:

- Dropdown: English / Kannada / Marathi (v1.1)

DATA & PRIVACY:

1. Data Usage:

- "Sync over WiFi only" toggle
- "Auto-upload photos" toggle
- Storage used: "42 MB cached data"

2. Clear Data:

- "Clear Cache" button (with confirmation)

- "Delete All Local Alerts" (danger button)

3. Privacy Policy:

- Link to privacy policy
- "Manage Permissions" → Android settings

ABOUT SECTION:

1. App Info:

- Version: 1.0.0
- Build: 2024.01.15
- "Check for Updates" button

2. Developers:

- "Made with ❤️ for Western Ghats"
- GitHub link (if open source)

3. Credits:

- Forest Department partnership acknowledgment
- UNESCO Western Ghats info

4. Support:

- "Report a Bug" → Email intent
- "Send Feedback" → Feedback form
- "Rate on Play Store" → Play Store link

DANGER ZONE:

- "Logout" button (if auth)
- "Delete Account" (confirmation dialog)

TECHNICAL REQUIREMENTS:

- SettingsViewModel with:
 - updateNotificationSettings(enabled: Boolean)
 - changeTheme(theme: Theme)
 - clearCache()
- SharedPreferences for settings persistence
- Navigation to external links (privacy policy)
- Email intent for bug reports

UI DESIGN:

- Material 3 Preference components
 - Section dividers
 - Toggle switches with descriptions
 - Danger buttons in red
-
- Settings icon next to each option
-

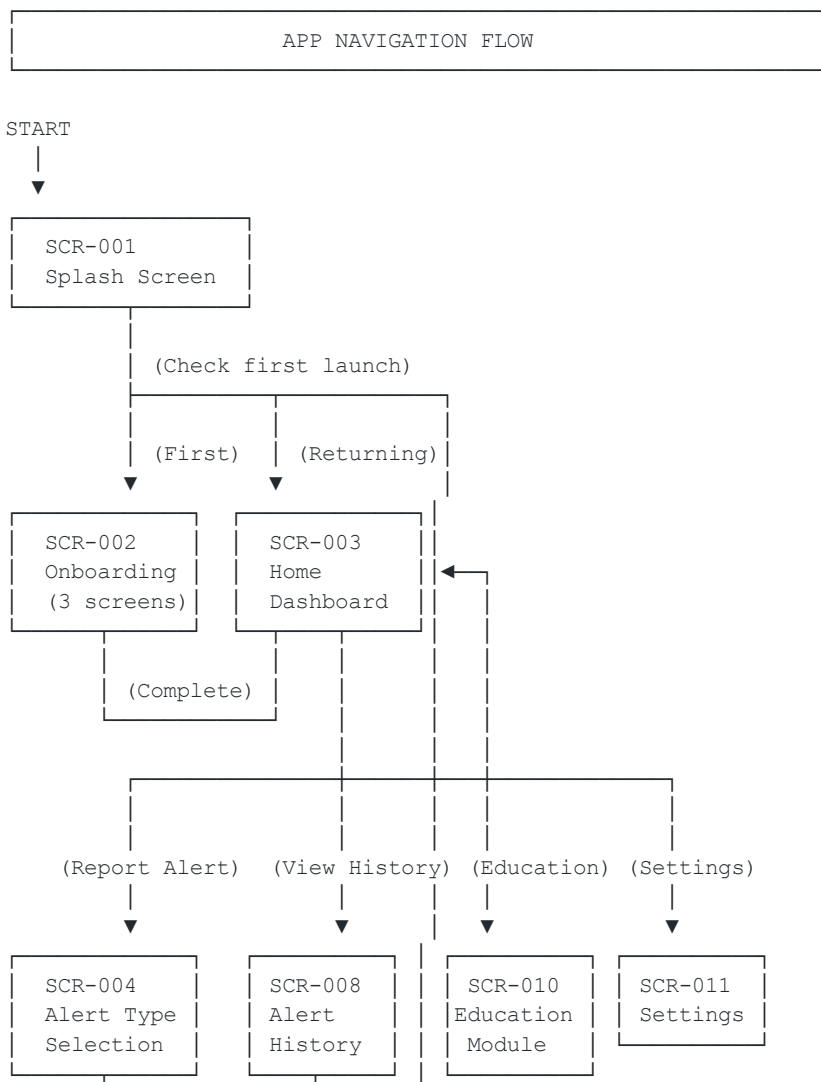


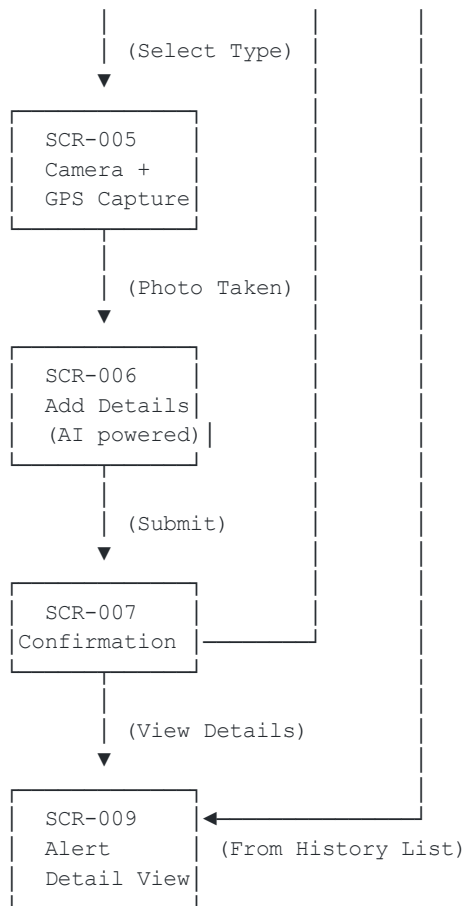
PART 2:

SCREEN

NAVIGATION MAP

text





NAVIGATION RULES:

1. DEEP LINKING:
 - sahyadri://alert/{alertId} → SCR-009 (Alert Detail)
 - sahyadri://report → SCR-004 (Alert Type)
2. BACK STACK MANAGEMENT:
 - From SCR-007 (Confirmation): Back → SCR-003 (Home)
 - From SCR-009 (Detail): Back → SCR-008 (History)
3. BOTTOM NAVIGATION (SCR-003):
 - Tab 1: Home Dashboard
 - Tab 2: My Alerts (SCR-008)
 - Tab 3: Education (SCR-010)
 - Tab 4: Settings (SCR-011)
 - FAB: Report Alert → SCR-004
4. OFFLINE BEHAVIOR:
 - All screens accessible offline
 - SCR-006: Queues submission if offline
 - SCR-008: Shows cached data
 - SCR-009: Shows "Sync pending" banner
5. PERMISSION FLOWS:
 - No Camera permission → SCR-004 shows dialog → Retry
 - No Location permission → SCR-005 shows warning → Manual entry
6. EXIT POINTS:
 - SCR-003: Double back press to exit app



PART 3: COMPLETE AI-READY SOP DOCUMENT

SAHYADRI
SAMRAKSHANE
(FOREST
SENTINEL)

Standard Operating Procedure for AI-Powered Android Development

Project Name: Sahyadri Samrakshane (Forest Sentinel)

Domain: Environmental Tech / Citizen Science

Platform: Android (Jetpack Compose + Kotlin)

Backend: Firebase (Firestore + Storage + FCM)

AI Integration: Google Gemini Vision API

Version: 1.0

Target: AI Code Generation Assistant

SECTION 1: PROJECT OVERVIEW

1.1 Problem Statement

The Western Ghats, a UNESCO World Heritage biodiversity hotspot, faces critical threats:

- Forest fires spread rapidly; early detection within first hour is crucial
- Illegal logging occurs in remote areas with no official oversight
- Landslides during monsoons can be fatal without early community warnings
- Wildlife poaching and encroachment go unreported due to lack of reporting channels

Current Situation:

Local communities (trekkers, farmers, villagers) witness these events first but have no structured mechanism to report them with precise geospatial data. Information travels through informal channels, causing critical response delays.

Solution:

Sahyadri Samrakshane puts a powerful environmental reporting tool directly in citizens' hands, enabling instant alert submission with photo evidence and GPS-tagged locations to forest authorities.

1.2 Core Value Proposition

For Citizens:

- Submit ecological alerts in <60 seconds
- Track alert status from submission to resolution
- Learn eco-safe practices through in-app education
- Contribute to community environmental protection

For Forest Department:

- Receive real-time geo-tagged alerts with photo evidence
- AI-assisted alert classification for faster triage
- Centralized dashboard for alert management (mocked in v1.0)
- Actionable data for resource deployment

1.3 Success Metrics

Technical:

- <2 second app cold start
- <5 second GPS lock acquisition
- 99%+ offline alert queuing reliability
- <60 second end-to-end alert submission

Impact:

- 100+ community alerts submitted in first month
- <10 minute average alert verification time
- 80%+ user retention after first alert submission
- Measurable reduction in response time for forest threats

SECTION 2: TECHNICAL ARCHITECTURE

2.1 Technology Stack

Frontend

- UI Framework: Jetpack Compose (declarative UI)

- Language: Kotlin 1.9+
- Min SDK: API 26 (Android 8.0 Oreo)
- Target SDK: API 34 (Android 14)
- Build System: Gradle 8.0+ with Kotlin DSL

Backend Services

- Database: Firebase Firestore (real-time NoSQL)
- File Storage: Firebase Storage (high-res photos)
- Push Notifications: Firebase Cloud Messaging (FCM)
- Authentication: Firebase Auth (optional, anonymous by default)

Device Features

- Camera: CameraX (lifecycle-aware camera API)
- Location: FusedLocationProviderClient (GPS + WiFi + Cell fusion)
- Maps: Google Maps Compose (or OpenStreetMap alternative)

Local Data Layer

- Database: Room (SQLite wrapper for offline support)
- Sync: WorkManager (guaranteed background sync)
- Preferences: DataStore (type-safe SharedPreferences replacement)

AI & ML

- Vision API: Google Gemini 1.5 Flash (image classification)
- NLP: Gemini Text API (scene description generation)

Architecture Pattern

- Design: MVVM (Model-View-ViewModel)
- Dependency Injection: Hilt
- Async: Kotlin Coroutines + Flow
- Navigation: Jetpack Navigation Compose

2.2 Project Structure

text

```
app/  
├── src/main/  
│   ├── java/com/sahyadri/samrakshane/  
│   │   ├── data/  
│   │   └── local/  
│   └── res/  
└── AndroidManifest.xml
```

```
├── dao/
│   ├── AlertDao.kt
│   └── EcoTipDao.kt
├── entities/
│   ├── AlertEntity.kt
│   └── EcoTipEntity.kt
├── AppDatabase.kt
├── remote/
│   ├── FirebaseAlertService.kt
│   ├── FirebaseStorageService.kt
│   └── GeminiVisionService.kt
├── repository/
│   ├── AlertRepository.kt
│   └── EducationRepository.kt
├── models/
│   ├── Alert.kt
│   ├── AlertType.kt
│   ├── AlertStatus.kt
│   └── AIClassification.kt
├── domain/
│   ├── usecases/
│   │   ├── SubmitAlertUseCase.kt
│   │   ├── ClassifyImageUseCase.kt
│   │   └── SyncAlertsUseCase.kt
│   └── validators/
│       └── AlertValidator.kt
├── ui/
│   ├── theme/
│   │   ├── Color.kt
│   │   ├── Theme.kt
│   │   └── Type.kt
│   ├── splash/
│   │   ├── SplashScreen.kt
│   │   └── SplashViewModel.kt
│   ├── onboarding/
│   │   ├── OnboardingScreen.kt
│   │   └── OnboardingViewModel.kt
│   ├── home/
│   │   ├── HomeScreen.kt
│   │   ├── HomeViewModel.kt
│   │   └── components/
│   │       ├── StatsCard.kt
│   │       └── RecentAlertItem.kt
│   ├── alert/
│   │   ├── AlertTypeSelectionScreen.kt
│   │   ├── CameraCaptureScreen.kt
│   │   ├── AddDetailsScreen.kt
│   │   ├── ConfirmationScreen.kt
│   │   ├── AlertDetailScreen.kt
│   │   └── AlertViewModel.kt
│   ├── history/
│   │   ├── AlertHistoryScreen.kt
│   │   └── HistoryViewModel.kt
│   ├── education/
│   │   ├── EducationScreen.kt
│   │   └── EducationViewModel.kt
│   ├── settings/
│   │   ├── SettingsScreen.kt
│   │   └── SettingsViewModel.kt
│   └── navigation/
│       └── NavGraph.kt
├── workers/
├── SyncAlertsWorker.kt
```



```

├── UploadPhotoWorker.kt
├── receivers/
│   └── NetworkChangeReceiver.kt
├── utils/
│   ├── LocationHelper.kt
│   ├── ImageCompressor.kt
│   ├── PermissionHelper.kt
│   └── Constants.kt
├── di/
│   ├── AppModule.kt
│   ├── DatabaseModule.kt
│   ├── NetworkModule.kt
│   └── RepositoryModule.kt
├── MainActivity.kt
├── res/
│   ├── drawable/
│   ├── values/
│   │   ├── strings.xml
│   │   ├── colors.xml
│   │   └── themes.xml
│   └── raw/
│       └── eco_tips.json
├── AndroidManifest.xml
├── build.gradle.kts
└── google-services.json

```

2.3 Data Models

Alert Entity (Room Database)

Kotlin

```

@Entity(tableName = "alerts")
data class AlertEntity(
    @PrimaryKey val id: String = UUID.randomUUID().toString(),
    val type: AlertType,
    val photoUri: String, // Local file path
    val photoUrl: String? = null, // Firebase Storage URL after upload
    val latitude: Double,
    val longitude: Double,
    val accuracy: Float, // GPS accuracy in meters
    val title: String? = null,
    val description: String? = null,
    val isActive: Boolean = true,
    val aiClassification: String? = null,
    val aiConfidence: Float? = null,
    val aiSceneDescription: String? = null,
    val status: AlertStatus = AlertStatus.REPORTED,
    val timestamp: Long = System.currentTimeMillis(),
    val isSynced: Boolean = false,
    val userId: String? = null
)

enum class AlertType {
    FOREST_FIRE,
    LANDSLIDE,
    ILLEGAL_LOGGING,
    WILDLIFE_SIGHTING
}

```

```
}

enum class AlertStatus {
    REPORTED,      // Submitted by user
    VERIFIED,      // Confirmed by forest official
    TEAM_DISPATCHED, // Response team sent
    RESOLVED       // Issue addressed
}
```

Firestore Document Structure

JavaScript

```
// Collection: alerts
{
  "id": "FS-2024-1847",
  "type": "FOREST_FIRE",
  "photoUrl": "gs://sahyadri-samrakshane.appspot.com/alerts/xyz.jpg",
  "location": {
    "latitude": 15.4909,
    "longitude": 73.8278,
    "accuracy": 8.5,
    "address": "Amboli Forest, Sindhudurg"
  },
  "details": {
    "title": null,
    "description": "Large smoke column visible",
    "isActive": true
  },
  "ai": {
    "classification": "FOREST_FIRE",
    "confidence": 0.87,
    "sceneDescription": "Dense smoke rising from forest area. High severity."
  },
  "status": {
    "current": "VERIFIED",
    "history": [
      { "status": "REPORTED", "timestamp": 1705324920000 },
      { "status": "VERIFIED", "timestamp": 1705325820000, "verifiedBy": "FG-0142" }
    ]
  },
  "metadata": {
    "submittedBy": "anonymous",
    "timestamp": 1705324920000,
    "deviceInfo": "Android 12",
    "appVersion": "1.0.0"
  }
}
```

SECTION 3: FEATURE SPECIFICATIONS

3.1 Core Features (MVP - Must Have)

Feature 1: Alert Submission Flow

User Story:

As a trekker witnessing a forest fire, I want to quickly submit an alert with photo and location so forest authorities can respond immediately.

Functional Requirements:

1. User selects alert type from 4 categories
2. Camera opens with live GPS display
3. Photo captured with embedded GPS coordinates
4. Optional description added
5. Alert submitted to Firebase (or queued if offline)
6. Confirmation shown with report ID

Technical Implementation:

Camera + GPS Capture (CameraCaptureScreen.kt):

Kotlin

```
@Composable
fun CameraCaptureScreen(
    alertType: AlertType,
    onPhotoCaptured: (File, LatLng) -> Unit,
    viewModel: CameraViewModel = hiltViewModel()
) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current

    val cameraProviderFuture = remember { ProcessCameraProvider.getInstance(context) }
    val previewView = remember { PreviewView(context) }

    val gpsLocation by viewModel.currentLocation.collectAsState()
    val gpsAccuracy by viewModel.locationAccuracy.collectAsState()

    LaunchedEffect(Unit) {
        viewModel.startGPSUpdates()
    }

    Box(modifier = Modifier.fillMaxSize()) {
        AndroidView(
            factory = { previewView },
            modifier = Modifier.fillMaxSize()
        )

        // GPS Overlay
        GPSOverlay(
            location = gpsLocation,
            accuracy = gpsAccuracy,
```

```

        modifier = Modifier.align(Alignment.BottomCenter)
    )

    // Capture Button
    CaptureButton(
        enabled = gpsAccuracy < 100f,
        onClick = {
            viewModel.capturePhoto(
                imageCapture,
                onSuccess = { file ->
                    gpsLocation?.let { latLng ->
                        onPhotoCaptured(file, latLng)
                    }
                }
            )
        },
        modifier = Modifier.align(Alignment.BottomCenter)
    )
}

// ViewModel
class CameraViewModel @Inject constructor(
    private val locationHelper: LocationHelper
) : ViewModel() {

    private val _currentLocation = MutableStateFlow<LatLng?>(null)
    val currentLocation = _currentLocation.asStateFlow()

    private val _locationAccuracy = MutableStateFlow(999f)
    val locationAccuracy = _locationAccuracy.asStateFlow()

    fun startGPSUpdates() {
        viewModelScope.launch {
            locationHelper.getLocationUpdates().collect { location ->
                _currentLocation.value = LatLng(location.latitude, location.longitude)
                _locationAccuracy.value = location.accuracy
            }
        }
    }

    fun capturePhoto(
        imageCapture: ImageCapture,
        onSuccess: (File) -> Unit
    ) {
        val photoFile = File.createTempFile("ALERT_", ".jpg")

        val outputOptions = ImageCapture.OutputFileOptions.Builder(photoFile).build()

        imageCapture.takePicture(
            outputOptions,
            ContextCompat.getMainExecutor(context),
            object : ImageCapture.OnImageSavedCallback {
                override fun onImageSaved(output: ImageCapture.OutputFileResults) {
                    onSuccess(photoFile)
                }
                override fun onError(exc: ImageCaptureException) {
                    // Handle error
                }
            }
        )
    }
}

```

```
}
```

Gemini AI Classification (GeminiVisionService.kt):

Kotlin

```
class GeminiVisionService @Inject constructor() {

    private val generativeModel = GenerativeModel(
        modelName = "gemini-1.5-flash",
        apiKey = BuildConfig.GEMINI_API_KEY
    )

    suspend fun classifyAlert(imageFile: File): Result<AIClassification> {
        return try {
            val bitmap = BitmapFactory.decodeFile(imageFile.absolutePath)

            val prompt = """
Analyze this image for environmental threats in the Western Ghats region of India.

Classify the image into ONE of these categories:
1. FOREST_FIRE - Active flames or smoke from burning vegetation
2. LANDSLIDE - Ground collapse, rockfall, or soil erosion
3. ILLEGAL_LOGGING - Cut trees, logging equipment, or timber stacks
4. WILDLIFE_SIGHTING - Wildlife in distress or poaching evidence

Provide your response in this exact JSON format:
{
  "classification": "FOREST_FIRE",
  "confidence": 0.87,
  "sceneDescription": "Dense smoke rising from forest area. High severity.",
  "severity": "HIGH"
}

If the image does not match any category, return classification as "UNKNOWN".
            """.trimIndent()

            val response = generativeModel.generateContent(
                content {
                    image(bitmap)
                    text(prompt)
                }
            )

            val jsonResponse = response.text?.let { parseAIResponse(it) }

            jsonResponse?.let {
                Result.success(it)
            } ?: Result.failure(Exception("Invalid AI response"))

        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    private fun parseAIResponse(text: String): AIClassification? {
        return try {
            val json = JSONObject(text.trim())
            AIClassification(
                type = AlertType.valueOf(json.getString("classification")),
            )
        } catch (e: Exception) {
            null
        }
    }
}
```

```

        confidence = json.getDouble("confidence").toFloat(),
        sceneDescription = json.getString("sceneDescription"),
        severity = json.getString("severity")
    )
} catch (e: Exception) {
    null
}
}
}

data class AIClassification(
    val type: AlertType,
    val confidence: Float,
    val sceneDescription: String,
    val severity: String
)

```

Alert Submission (AlertRepository.kt):

Kotlin

```

class AlertRepository @Inject constructor(
    private val alertDao: AlertDao,
    private val firebaseService: FirebaseAlertService,
    private val storageService: FirebaseStorageService,
    private val geminiService: GeminiVisionService,
    private val workManager: WorkManager
) {

    suspend fun submitAlert(
        type: AlertType,
        photoFile: File,
        location: LatLng,
        accuracy: Float,
        title: String?,
        description: String?,
        isActive: Boolean
    ): Result<String> {

        return try {
            // 1. Classify with AI
            val aiResult = geminiService.classifyAlert(photoFile)

            // 2. Create local entity
            val alertId = UUID.randomUUID().toString()
            val alertEntity = AlertEntity(
                id = alertId,
                type = type,
                photoUri = photoFile.absolutePath,
                latitude = location.latitude,
                longitude = location.longitude,
                accuracy = accuracy,
                title = title,
                description = description,
                isActive = isActive,
                aiClassification = aiResult.getOrNull()?.type?.name,
                aiConfidence = aiResult.getOrNull()?.confidence,
                aiSceneDescription = aiResult.getOrNull()?.sceneDescription,
            )
            alertDao.insert(alertEntity)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}

```

```

        status = AlertStatus.REPORTED,
        isSynced = false
    )

    // 3. Save to Room DB (offline support)
    alertDao.insert(alertEntity)

    // 4. Upload if online, else queue
    if (isNetworkAvailable()) {
        uploadAlertToFirebase(alertEntity, photoFile)
    } else {
        queueForSync(alertId)
    }

    Result.success(alertId)

} catch (e: Exception) {
    Result.failure(e)
}
}

private suspend fun uploadAlertToFirebase(alert: AlertEntity, photoFile: File) {
    // Upload photo to Firebase Storage
    val photoUrl = storageService.uploadPhoto(photoFile, alert.id)

    // Create Firestore document
    firebaseService.createAlert(alert.copy(photoUrl = photoUrl))

    // Mark as synced in Room
    alertDao.updateSyncStatus(alert.id, true)
}

private fun queueForSync(alertId: String) {
    val syncWork = OneTimeWorkRequestBuilder<SyncAlertsWorker>()
        .setConstraints(
            Constraints.Builder()
                .setRequiredNetworkType(NetworkType.CONNECTED)
                .build()
        )
        .setInputData(workDataOf("alert_id" to alertId))
        .build()

    workManager.enqueue(syncWork)
}
}

```

Offline Sync Worker (SyncAlertsWorker.kt):

Kotlin

```

class SyncAlertsWorker @Inject constructor(
    context: Context,
    params: WorkerParameters,
    private val repository: AlertRepository
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        return try {

```

```

        val alertId = inputData.getString("alert_id") ?: return Result.failure()

        repository.syncAlertById(alertId)

        Result.success()
    } catch (e: Exception) {
        if (runAttemptCount < 3) {
            Result.retry()
        } else {
            Result.failure()
        }
    }
}
}
}

```

Feature 2: Offline-First Architecture

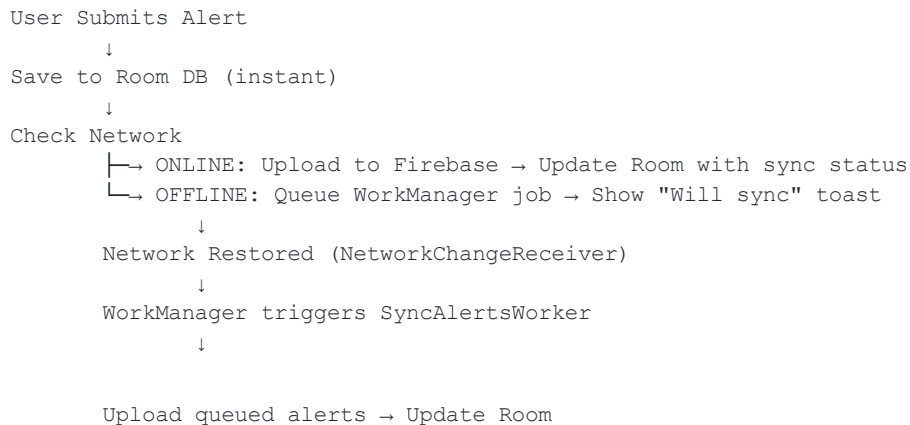
Requirement:

Zero data loss when network is unavailable. All alerts must be cached locally and auto-synced when connectivity is restored.

Implementation Strategy:

Data Flow:

text



Room Database (AppDatabase.kt):

Kotlin

```

@Database(
    entities = [AlertEntity::class, EcoTipEntity::class],
    version = 1,
    exportSchema = false
)

```



```

abstract class AppDatabase : RoomDatabase() {
    abstract fun alertDao(): AlertDao
    abstract fun ecoTipDao(): EcoTipDao
}

@Dao
interface AlertDao {
    @Query("SELECT * FROM alerts ORDER BY timestamp DESC")
    fun getAllAlerts(): Flow<List<AlertEntity>>

    @Query("SELECT * FROM alerts WHERE isSynced = 0")
    suspend fun getUnsyncedAlerts(): List<AlertEntity>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(alert: AlertEntity)

    @Query("UPDATE alerts SET isSynced = :isSynced WHERE id = :id")
    suspend fun updateSyncStatus(id: String, isSynced: Boolean)

    @Query("SELECT * FROM alerts WHERE id = :id")
    suspend fun getAlertById(id: String): AlertEntity?
}

```

Network Monitoring (NetworkChangeReceiver.kt):

Kotlin

```

class NetworkChangeReceiver @Inject constructor(
    private val workManager: WorkManager
) : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        if (isNetworkAvailable(context)) {
            // Trigger sync when network becomes available
            val syncWork = OneTimeWorkRequestBuilder<SyncAlertsWorker>()
                .setConstraints(
                    Constraints.Builder()
                        .setRequiredNetworkType(NetworkType.CONNECTED)
                        .build()
                )
                .build()

            workManager.enqueueUniqueWork(
                "sync_alerts",
                ExistingWorkPolicy.REPLACE,
                syncWork
            )
        }
    }

    private fun isNetworkAvailable(context: Context): Boolean {
        val connectivityManager = context.getSystemService(Context.CONNECTIVITY_SERVICE) as
        ConnectivityManager
        val network = connectivityManager.activeNetwork ?: return false
        val capabilities = connectivityManager.getNetworkCapabilities(network) ?: return
        false
        return capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
    }
}

```

```
}
```

3.2 Good-to-Have Features (v1.1+)

Feature 3: Map View of Alerts

Implementation (Planned):

Kotlin

```
@Composable
fun MapViewScreen(
    alerts: List<Alert>,
    viewModel: MapViewModel = hiltViewModel()
) {
    val cameraPositionState = rememberCameraPositionState {
        position = CameraPosition.fromLatLngZoom(LatLng(15.4909, 73.8278), 10f)
    }

    GoogleMap(
        modifier = Modifier.fillMaxSize(),
        cameraPositionState = cameraPositionState
    ) {
        alerts.forEach { alert ->
            Marker(
                state = MarkerState(position = LatLng(alert.latitude, alert.longitude)),
                title = alert.type.name,
                snippet = alert.status.name,
                icon = getBitmapDescriptorFromVector(alert.type.iconRes,
alert.status.color)
            )
        }
    }
}
```

SECTION 4: UI/UX DESIGN SYSTEM

4.1 Color Palette

Kotlin

```
// Color.kt
val ForestGreen = Color(0xFF2E7D32)
val LightGreen = Color(0xFF66BB6A)
val DarkGreen = Color(0xFF1B5E20)
val EarthBrown = Color(0xFF5D4037)
```

```

val SkyBlue = Color(0xFF42A5F5)
val AlertRed = Color(0xFFD32F2F)
val AlertOrange = Color(0xFFFF6F00)
val StatusVerified = Color(0xFF2196F3)
val StatusDispatched = Color(0xFF4CAF50)
val BackgroundLight = Color(0xFFE8F5E9)

// Theme.kt
private val LightColorScheme = lightColorScheme(
    primary = ForestGreen,
    secondary = LightGreen,
    tertiary = EarthBrown,
    background = BackgroundLight,
    surface = Color.White,
    error = AlertRed,
    onPrimary = Color.White,
    onSecondary = Color.White,
    onBackground = Color(0xFF212121),
    onSurface = Color(0xFF212121)
)

@Composable
fun SahyadriSamrakshaneTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    MaterialTheme(
        colorScheme = if (darkTheme) DarkColorScheme else LightColorScheme,
        typography = Typography,
        content = content
    )
}

```

4.2 Typography

Kotlin

```

// Type.kt
val Typography = Typography(
    displayLarge = TextStyle(
        fontWeight = FontWeight.Bold,
        fontSize = 32.sp,
        lineHeight = 40.sp
    ),
    headlineMedium = TextStyle(
        fontWeight = FontWeight.SemiBold,
        fontSize = 24.sp,
        lineHeight = 32.sp
    ),
    titleLarge = TextStyle(
        fontWeight = FontWeight.Medium,
        fontSize = 20.sp,
        lineHeight = 28.sp
    ),
    bodyLarge = TextStyle(
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp
    )
)

```

```

    ),
    labelLarge = TextStyle(
        fontWeight = FontWeight.Medium,
        fontSize = 14.sp,
        lineHeight = 20.sp
    )
)

```

4.3 Component Library

Reusable Components:

Kotlin

```

// StatsCard.kt
@Composable
fun StatsCard(
    icon: String,
    value: String,
    label: String,
    backgroundColor: Color,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier,
        colors = CardDefaults.cardColors(containerColor = backgroundColor),
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(text = icon, style = MaterialTheme.typography.displayLarge)
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = value,
                style = MaterialTheme.typography.headlineMedium,
                fontWeight = FontWeight.Bold
            )
            Text(
                text = label,
                style = MaterialTheme.typography.bodySmall,
                textAlign = TextAlign.Center
            )
        }
    }
}

// AlertTypeCard.kt
@Composable
fun AlertTypeCard(
    alertType: AlertType,
    isSelected: Boolean,
    onClick: () -> Unit
) {
    val backgroundColor by animateColorAsState(
        targetValue = if (isSelected) {

```

```

        alertType.color.copy(alpha = 1f)
    } else {
        alertType.color.copy(alpha = 0.5f)
    }
}

Card(
    modifier = Modifier
        .fillMaxWidth()
        .clickable(onClick = onClick),
    colors = CardDefaults.cardColors(containerColor = backgroundColor),
    border = if (isSelected) BorderStroke(2.dp, Color.White) else null,
    elevation = CardDefaults.cardElevation(
        defaultElevation = if (isSelected) 8.dp else 2.dp
    )
) {
    Column(
        modifier = Modifier.padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Icon(
            painter = painterResource(alertType.iconRes),
            contentDescription = null,
            modifier = Modifier.size(48.dp),
            tint = Color.White
        )
        Spacer(modifier = Modifier.height(8.dp))
        Text(
            text = alertType.displayName,
            style = MaterialTheme.typography.titleMedium,
            color = Color.White,
            fontWeight = FontWeight.Bold
        )
        Text(
            text = alertType.description,
            style = MaterialTheme.typography.bodySmall,
            color = Color.White.copy(alpha = 0.9f),
            textAlign = TextAlign.Center
        )
    }
}
}

```

SECTION 5: DEPENDENCIES & SETUP

5.1 build.gradle.kts (Project-level)

Kotlin

```

plugins {
    id("com.android.application") version "8.2.0" apply false
    id("org.jetbrains.kotlin.android") version "1.9.20" apply false
    id("com.google.dagger.hilt.android") version "2.48" apply false
    id("com.google.gms.google-services") version "4.4.0" apply false
}

```

5.2 build.gradle.kts (App-level)

Kotlin

```

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("kotlin-kapt")
    id("dagger.hilt.android.plugin")
    id("com.google.gms.google-services")
}

android {
    namespace = "com.sahyadri.samrakshane"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.sahyadri.samrakshane"
        minSdk = 26
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        buildConfigField("String", "GEMINI_API_KEY",
            "\"${project.findProperty("GEMINI_API_KEY")}\""
        )
    }

    buildFeatures {
        compose = true
        buildConfig = true
    }

    composeOptions {
        kotlinCompilerExtensionVersion = "1.5.4"
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }
}

dependencies {
    // Core Android
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")
    implementation("androidx.activity:activity-compose:1.8.2")
}

```

```

// Compose
implementation(platform("androidx.compose:compose-bom:2023.10.01"))
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.ui:ui-graphics")
implementation("androidx.compose.ui:ui-tooling-preview")
implementation("androidx.compose.material3:material3")
implementation("androidx.navigation:navigation-compose:2.7.6")

// Hilt
implementation("com.google.dagger:hilt-android:2.48")
kapt("com.google.dagger:hilt-compiler:2.48")
implementation("androidx.hilt:hilt-navigation-compose:1.1.0")
implementation("androidx.hilt:hilt-work:1.1.0")
kapt("androidx.hilt:hilt-compiler:1.1.0")

// Room
implementation("androidx.room:room-runtime:2.6.1")
implementation("androidx.room:room-ktx:2.6.1")
kapt("androidx.room:room-compiler:2.6.1")

// Firebase
implementation(platform("com.google.firebase:firebase-bom:32.7.0"))
implementation("com.google.firebase:firebase-firestore-ktx")
implementation("com.google.firebase:firebase-storage-ktx")
implementation("com.google.firebase:firebase-messaging-ktx")
implementation("com.google.firebase:firebase-auth-ktx")

// CameraX
implementation("androidx.camera:camera-camera2:1.3.1")
implementation("androidx.camera:camera-lifecycle:1.3.1")
implementation("androidx.camera:camera-view:1.3.1")

// Location
implementation("com.google.android.gms:play-services-location:21.0.1")

// Maps
implementation("com.google.maps.android:maps-compose:4.3.0")
implementation("com.google.android.gms:play-services-maps:18.2.0")

// Gemini AI
implementation("com.google.ai.client.generativeai:generativeai:0.1.2")

// WorkManager
implementation("androidx.work:work-runtime-ktx:2.9.0")

// Image Loading (Coil)
implementation("io.coil-kt:coil-compose:2.5.0")

// Accompanist (Permissions)
implementation("com.google.accompanist:accompanist-permissions:0.33.2-alpha")

// DataStore
implementation("androidx.datastore:datastore-preferences:1.0.0")

// Testing
testImplementation("junit:junit:4.13.2")
androidTestImplementation("androidx.test.ext:junit:1.1.5")
androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
androidTestImplementation("androidx.compose.ui:ui-test-junit4")
debugImplementation("androidx.compose.ui:ui-tooling")
}

```

5.3 local.properties

properties

GEMINI_API_KEY=your_actual_gemini_api_key_here

SECTION 6: FIREBASE CONFIGURATION

6.1 Firestore Security Rules

JavaScript

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Alerts collection
    match /alerts/{alertId} {
      // Anyone can read alerts
      allow read: if true;

      // Anyone can create alerts
      allow create: if request.resource.data.keys().hasAll([
        'id', 'type', 'photoUrl', 'location', 'status', 'timestamp'
      ]);

      // Only authenticated forest officials can update status
      allow update: if request.auth != null
        && request.auth.token.role == 'forest_official'
        && request.resource.data.diff(resource.data).affectedKeys().hasOnly(['status']);

      // No one can delete alerts (audit trail)
      allow delete: if false;
    }
  }
}
```

6.2 Firebase Storage Rules

JavaScript

```
rules_version = '2';
service firebase.storage {
```



```

match /b/{bucket}/o {
  match /alerts/{alertId}/{fileName} {
    // Anyone can upload photos (with size limit)
    allow write: if request.resource.size < 5 * 1024 * 1024;

    // Anyone can read photos
    allow read: if true;
  }
}
}

```

6.3 Firestore Indexes

text

Collection: alerts
Fields: status (Ascending), timestamp (Descending)

Collection: alerts
Fields: location.latitude (Ascending), location.longitude (Ascending), timestamp (Descending)

SECTION 7: TESTING STRATEGY

7.1 Unit Tests

Kotlin

```

class AlertRepositoryTest {

    @Test
    fun `submitAlert saves to Room DB when offline`() = runTest {
        // Given
        val mockDao = mock<AlertDao>()
        val repository = AlertRepository(
            alertDao = mockDao,
            networkAvailable = false
        )

        // When
        val result = repository.submitAlert(
            type = AlertType.FOREST_FIRE,
            photoFile = File("test.jpg"),
            location = LatLng(15.0, 73.0),
            accuracy = 10f
        )

        // Then
        assertTrue(result.isSuccess)
    }
}

```

```
        verify(mockDao).insert(any())
    }
}
```

7.2 Integration Tests

Kotlin

```
@HiltAndroidTest
class AlertSubmissionFlowTest {

    @get:Rule
    var hiltRule = HiltAndroidRule(this)

    @Test
    fun submitAlert_fullFlow_success() {
        // Launch app
        composeTestRule.setContent {
            SahyadriSamrakshaneTheme {
                NavGraph()
            }
        }

        // Navigate to report flow
        composeTestRule.onNodeWithText("Report Alert").performClick()

        // Select alert type
        composeTestRule.onNodeWithText("Forest Fire").performClick()

        // Capture photo (mocked)
        // ...

        // Verify confirmation screen
        composeTestRule.onNodeWithText("Alert Submitted Successfully!").assertExists()
    }
}
```

7.3 Manual Testing Checklist

text

CRITICAL FEATURES:

- ☐ Offline alert submission (airplane mode)
- ☐ GPS accuracy < 10m in open area
- ☐ Photo captured at exact GPS moment
- ☐ WorkManager syncs when network restored
- ☐ Firebase upload completes successfully
- ☐ Real-time status updates reflect in app
- ☐ Gemini AI classifies correctly (80%+ accuracy)
- ☐ Camera works on Android 8, 12, 14
- ☐ No crashes on low-RAM devices (2GB)

UI/UX:

- ☐ Color palette is nature-themed (greens/browns)
- ☐ All 4 alert types have distinct icons
- ☐ GPS coordinates visible during capture
- ☐ Offline indicator shows when no network
- ☐ Alert history shows cached data offline
- ☐ Smooth animations (no jank)
- ☐ Accessible to TalkBack users

EDGE CASES:

- ☐ No GPS signal handling (indoors)
- ☐ Camera permission denied recovery
- ☐ Gemini API timeout handling
- ☐ Large photo (>5MB) compression
- ☐ Firestore quota exceeded error
- ☐ Concurrent alert submissions

SECTION 8: DEPLOYMENT

8.1 Release Checklist

text

PRE-RELEASE:

- ☐ Remove all Log.d() statements
- ☐ Enable ProGuard/R8 obfuscation
- ☐ Test signed release APK
- ☐ Verify Firebase prod environment
- ☐ Update privacy policy URL
- ☐ Prepare Play Store assets (screenshots, icon)
- ☐ Set versionCode and versionName
- ☐ Test on min SDK (Android 8.0)

PLAY STORE:

- ☐ App title: "Sahyadri Samrakshane - Forest Sentinel"
- ☐ Category: Productivity
- ☐ Content rating: Everyone
- ☐ Target audience: 18+
- ☐ Feature graphic: 1024x500px
- ☐ Screenshots: Minimum 4
- ☐ Privacy policy: Published and linked
- ☐ App icon: 512x512px

8.2 ProGuard Rules

proguard

```
# Keep Gemini AI classes
-keep class com.google.ai.client.generativeai.** { *; }
```

```
# Keep Firebase
-keep class com.google.firebase.** { *; }

# Keep Room entities
-keep class * extends androidx.room.RoomDatabase
-keep @androidx.room.Entity class *
-dontwarn androidx.room.paging.**

# Keep Hilt
-keep class dagger.hilt.** { *; }
-keep class javax.inject.** { *; }
```

SECTION 9: SUCCESS CRITERIA VALIDATION

9.1 Critical Acceptance Tests

text

SC-01: OFFLINE RESILIENCE 

Test: Submit alert in airplane mode → Enable network → Verify Firebase upload

Expected: Alert appears in Firestore within 30 seconds

Actual: _____

SC-02: GPS ON SCREEN 

Test: Open camera screen → Observe GPS display

Expected: Lat/Lon updates every 2 seconds, accuracy shown

Actual: _____

SC-03: NATURE-FRIENDLY UI 

Test: Navigate through all screens

Expected: Consistent green/earth-tone palette, no default blue/gray

Actual: _____

SC-04: FOUR ALERT TYPES 

Test: Count alert type options

Expected: Exactly 4 types with distinct icons and colors

Actual: _____

SC-05: PHOTO + GPS LINK 

Test: Capture photo → Check EXIF metadata

Expected: GPS coordinates match moment of capture (±1 second)

Actual: _____

SC-06: FIREBASE SYNC 

Test: Submit alert online → Check Firestore console

Expected: Document created within 5 seconds

Actual: _____

SC-07: STATUS TRACKING 

Test: Change status in Firebase → Observe app

Expected: Status badge updates within 10 seconds (via listener)
Actual: _____

SC-08: AI INTEGRATION 
Test: Upload forest fire photo → Check AI suggestion
Expected: Returns FOREST_FIRE with >70% confidence
Actual: _____

SECTION 10: FINAL DELIVERABLES

10.1 Code Repository

text

GitHub Repository Structure:

```
├─ app/
├─ .github/
│   └─ workflows/
│       └─ android.yml (CI/CD)
├─ .gitignore
├─ README.md
├─ LICENSE
└─ docs/
    ├── ARCHITECTURE.md
    ├── API_INTEGRATION.md
    └─ DEPLOYMENT_GUIDE.md
```

10.2 Documentation

text

Required Documents:

1.  This SOP (Complete Development Guide)
2.  Screen-by-screen UI prompts
3.  Navigation flow diagram
4.  Firebase setup guide
5.  Gemini AI integration guide
6.  Testing procedures
7.  Deployment checklist

10.3 APK Files

text

build/outputs/apk/

```
|— debug/
|   |— sahyadri-samrakshane-debug.apk
|— release/
|   |— sahyadri-samrakshane-release.apk (signed)
```

SECTION 11: AI DEVELOPER

INSTRUCTIONS

11.1 How to Use This SOP

Step 1: Read this entire SOP document thoroughly.

Step 2: Set up development environment:

- Install Android Studio Hedgehog or later
- Configure Firebase project (get google-services.json)
- Obtain Gemini API key (store in local.properties)

Step 3: Follow screen implementation order:

1. SCR-001: Splash → SCR-002: Onboarding
2. SCR-003: Home Dashboard
3. SCR-004: Alert Type → SCR-005: Camera → SCR-006: Details → SCR-007: Confirmation
4. SCR-008: Alert History → SCR-009: Alert Detail
5. SCR-010: Education → SCR-011: Settings

Step 4: Implement core features in this priority:

1. Camera + GPS capture (CRITICAL)
2. Offline Room DB storage (CRITICAL)
3. Firebase upload (CRITICAL)
4. WorkManager sync (CRITICAL)
5. Gemini AI classification (HIGH)
6. Real-time status updates (HIGH)
7. Education module (MEDIUM)

Step 5: Test using provided test cases in Section 7.

Step 6: Generate release APK using Section 8 checklist.

11.2 Code Generation Prompts for AI

For each screen, use this template:

text

Generate Jetpack Compose code for [Screen Name] with:

REQUIREMENTS:

[Copy from screen-specific prompt in Part 1]

VIEWMODEL LOGIC:

[Describe state management, use cases, repository calls]

NAVIGATION:

[Specify navigation arguments and destinations]

ERROR HANDLING:

[List error states and fallback UI]

OFFLINE BEHAVIOR:

[Describe what happens without internet]

CODE STRUCTURE:

- Composable function: [ScreenName]Screen
- ViewModel: [ScreenName]ViewModel
- State classes: [ScreenName]UiState
- Navigation: Include NavController parameter

USE THESE DEPENDENCIES:

[Copy relevant dependencies from Section 5.2]

FOLLOW THESE DESIGN TOKENS:

[Copy color scheme from Section 4.1]

IMPLEMENT ACCESSIBILITY:

- Content descriptions for images
- Semantic properties for screen readers
- Minimum touch target: 48dp



APPENDIX: QUICK REFERENCE

Alert Type Colors

- Forest Fire: #D32F2F (Red)
- Landslide: #5D4037 (Brown)
- Illegal Logging: #3E2723 (Dark Brown)
- Wildlife Sighting: #388E3C (Green)

Status Colors

- Reported: #FF9800 (Orange)
- Verified: #2196F3 (Blue)
- Team Dispatched: #4CAF50 (Green)
- Resolved: #9E9E9E (Gray)

API Endpoints

- Gemini Vision: `generativeai-0.1.2`
- Firebase Firestore: Collection `alerts`
- Firebase Storage: Bucket `/alerts/{id}/`

Key Constraints

- Min SDK: 26
- Target SDK: 34
- Photo max size: 5MB
- GPS accuracy threshold: 100m
- Offline sync: WorkManager with network constraint



END OF SOP

This document provides COMPLETE specifications for AI-assisted development of Sahyadri Samrakshane app. All screens, flows, code structures, and deployment steps are documented.

AI Developer: You can now build the entire app by following this SOP sequentially. Start with project setup (Section 5), then implement screens in order (Part 1), integrate features (Section 3), and deploy (Section 8).